

GitHub Actions

GitHub Actions is a popular tool used in DevOps. It helps in implementing continuous integration and continuous deployment (CI/CD) besides other automation.

CI aids teams in promptly identifying and resolving issues that arise early in the development process. For example, every time a developer submits a code change to the GitHub repository, a CI process builds and tests the application immediately. Such a CI process can be built using GitHub Actions.

Components of GitHub Actions

Let's peek under the hood of GitHub Actions and discover how its various parts work together.

Workflow: A workflow is an automation pipeline for a given GitHub repository. It is defined in a YAML file under the '.github/workflows' folder. A workflow consists of one or more jobs triggered by repository events.

Event: An event signifies a change in the repository's status. For example, an event occurs when someone pushes code into a repository or makes a pull request, etc. A workflow can be configured to run against one or more such events. Moreover, a successful workflow can also trigger another workflow.

Job: A job is a sequence of steps to be executed such as scripts or other pre-defined actions. Multiple jobs can be defined in a workflow. The jobs in a workflow run independently in parallel unless they depend on each other. A job succeeds only if all the steps in it are executed without errors. Similarly, a workflow succeeds only if all the jobs defined in it get executed successfully.

Action: To reduce the complexity of defining workflows GitHub offers pre-defined actions. For example, there are actions available to set up a Node environment, or a Java environment. The large community of GitHub Actions also offers several useful actions through the marketplace. All you need

to do is correctly configure them. You can also define your own custom actions based on specific requirements.

Runners: We need servers to execute the jobs of workflows. These servers are called runners. By default, the jobs run on GitHub-hosted runners. They are available on Linux, macOS, and Windows platforms. We can also configure our own runners for special purposes such as safety, security, capacity, etc.

A use case for GitHub Actions

Let us consider the case of creating a Docker container image from a Java project managed by Maven. As you know, Maven handles dependency management apart from compilation, building, testing, and packaging. A Docker container is a self-contained image that can run as it is on Docker runtime.

Here we will automate a CI process to build and push the Docker image whenever someone pushes the code into the repository.

Set up a workflow: To create a workflow, follow these steps:

- Log in to GitHub, visit the repository of the code, click the 'Actions' menu item and press the 'New workflow' button.
- Search for 'Simple workflow' and click on 'Configure'. It creates a YAML file and displays the default workflow in the editor.
- Replace the code in the YAML with the following code snippet:

```
name: CI Workflow
on:
  pull_request:
    types: [opened, reopened]
    branches:
      - main
  push:
    branches:
      - main
jobs:
```

```
build:
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
  steps:
    - name: Checkout Repository
      uses: actions/checkout@v2
    - name: Set up JDK 8
      uses: actions/setup-java@v2
      with:
        java-version: '8'
        distribution: 'temurin'
    - name: Build with Maven
      run: |
        mvn -B package --file pom.xml
    - name: Build and Push Docker
      uses: mr-smithers-excellent/
docker-build-push@v5
      with:
        image: glarimy/glarimy-
assessments-api
        tags: latest
        registry: docker.io
        dockerfile: Dockerfile
        username: ${SECRETS.DOCKER_
ID}}
        password: ${SECRETS.DOCKER_
PWD}}
```

The name of the workflow is CI Workflow. The code inside 'on' defines when the workflow will trigger. This workflow is configured to be triggered when someone makes a pull request to the main branch, or someone pushes a file in the main branch.

In 'jobs' there is only one job named 'build'. To run a job, you need a runner. The runner can be a GitHub-hosted runner or a self-hosted runner. In the code snippet, we used the GitHub-hosted runner ubuntu-latest. The permissions to the job come next.

Let's now define the steps to be followed by the job for CI workflow. In our case, there are four steps.

Step 1: Check out the repository on the runner. We use a pre-defined action for this purpose.

Continued to page...51